

Практика 7 – Массивы. Удаление, вставка и сортировка элементов

Задание 1. Разбор удаления элемента из массива (пошагово)

Возьмём пример из лекции — удаление m -го элемента из массива.

Шаг 1. Понимание задачи

Дан массив X из n элементов. Нужно удалить элемент с номером m . После удаления массив должен содержать $n-1$ элемент, а все элементы, которые были после удалённого, сдвигаются влево на одну позицию.

Шаг 2. Как выглядит сдвиг на примере

Пусть есть массив из 6 элементов: [10, 20, 30, 40, 50, 60]
Удаляем 3-й элемент (значение 30).

Алгоритм удаления:

- $X[3] = X[4] \rightarrow$ на место 30 ставим 40 $\rightarrow$ [10, 20, 40, 40, 50, 60]
- $X[4] = X[5] \rightarrow$ на место второго 40 ставим 50 $\rightarrow$ [10, 20, 40, 50, 50, 60]
- $X[5] = X[6] \rightarrow$ на место второго 50 ставим 60 $\rightarrow$ [10, 20, 40, 50, 60, 60]

Теперь последние две позиции дублируются, но мы знаем, что размер массива уменьшился. Фактически мы работаем только с первыми 5 элементами: [10, 20, 40, 50, 60].

Шаг 3. Общий алгоритм для удаления m -го элемента

Для удаления элемента с номером m из массива размером n нужно:

1. Запустить цикл от $i = m$ до $n-1$
2. На каждом шаге выполнить $X[i] = X[i+1]$
3. Уменьшить n на 1

Код цикла:

```
for i := m to n-1 do
```

```
  X[i] := X[i+1];
```

```
n := n - 1;
```

Шаг 4. Что происходит с индексами после удаления

После удаления элемента:

- Количество элементов уменьшается на 1
- Элементы, которые были после удалённого, получают новые индексы (на один меньше)
- Например, элемент, который был на позиции $m+1$, становится на позицию m

Шаг 5. Важное замечание при удалении в цикле

Если вы удаляете элементы в цикле (например, все отрицательные), после удаления текущего элемента не нужно увеличивать индекс. Потому что на место удалённого пришёл следующий элемент, и его нужно проверить на следующей итерации.

Пример неправильного кода:

```
for i := 1 to n do
```

```
  if X[i] < 0 then
```

```
    begin
```

```

    for j := i to n-1 do
        X[j] := X[j+1];
    n := n - 1;
    i := i + 1; // ОШИБКА! Не нужно увеличивать i
end;

```

Правильно: после удаления не делать $i := i + 1$, а оставить i как есть.

Задание 2. Разбор программы удаления отрицательных элементов из лекции

Возьмём программу из лекции — удаление всех отрицательных элементов.

Код программы:

```

program upor_massiv;
var
    i, n, j : byte;
    X: array [1..100] of real;
begin
    writeln('введите размер массива');
    readln(n);

    for i := 1 to n do
    begin
        write('X[' , i, ']= ');
        readln(X[i]);
    end;

    writeln('массив X');
    for i := 1 to n do
        write(x[i]:5:2, ' ');
    writeln;

    i := 1;
    while (i <= n) do
    begin
        if x[i] < 0 then
        begin
            for j := i to n-1 do
                x[j] := x[j+1];
            n := n - 1;
        end
        else
            i := i + 1;
        end;

        writeln('Изменённый массив');
    end;

```

```

for i := 1 to n do
  write(X[i]:5:2, ' ');
writeln;
end.

```

Шаг 1. Почему используется While, а не For

В программе используется цикл while, потому что количество элементов массива меняется во время выполнения. В цикле for трудно управлять счётчиком, когда размер массива уменьшается.

Шаг 2. Разберём работу алгоритма на конкретном примере

Пусть массив: [5, -3, 2, -1, 4], n = 5

i = 1: X[1]=5 >=0 → i увеличивается до 2

i = 2: X[2]=-3 <0 → удаляем

- Сдвиг: X[2]=X[3]=2, X[3]=X[4]=-1, X[4]=X[5]=4
- Массив стал: [5, 2, -1, 4, 4], n = 4
- i не увеличивается (остаётся 2)

i = 2: X[2]=2 >=0 → i увеличивается до 3

i = 3: X[3]=-1 <0 → удаляем

- Сдвиг: X[3]=X[4]=4
- Массив стал: [5, 2, 4, 4, 4], n = 3
- i не увеличивается (остаётся 3)

i = 3: X[3]=4 >=0 → i увеличивается до 4

i = 4, но n = 3, условие i <= n (4 <= 3) ложно → выход из цикла

Результат: [5, 2, 4]

Шаг 3. Почему нельзя увеличивать i после удаления

Если бы после удаления мы увеличили i, то пропустили бы элемент, который сдвинулся на место удалённого. В примере выше, после удаления -3 на его место пришло 2. Если бы мы увеличили i, то перешли бы к индексу 3 и пропустили бы проверку числа 2.

Задание 3. Создай программу удаления элементов самостоятельно

Условие: Написать программу, которая запрашивает у пользователя массив из n целых чисел (n вводит пользователь, но не более 50). Затем программа удаляет все элементы, равные заданному числу k, и выводит получившийся массив.

Выполни задание по шагам.

Шаг 1. Заголовок и переменные

Напиши:

```

Program DeleteElements;
var
  X: array [1..50] of integer;
  i, j, n, k: integer;

```

Шаг 2. Начало и ввод данных

Напиши:

```

Begin

```

```

write('Введите количество элементов (не более 50): ');
readln(n);
writeln('Введите элементы массива:');
for i := 1 to n do
  readln(X[i]);
write('Введите число k для удаления: ');
readln(k);

```

Шаг 3. Удаление элементов, равных k
 Напиши:

```

i := 1;
while i <= n do
begin
  if X[i] = k then
  begin
    for j := i to n-1 do
      X[j] := X[j+1];
    n := n - 1;
  end
  else
    i := i + 1;
  end;
end;

```

Шаг 4. Вывод результата
 Напиши:

```

writeln('Массив после удаления элементов ', k, ':');
if n = 0 then
  writeln('Массив пуст')
else
  for i := 1 to n do
    write(X[i], ' ');
  writeln;
  readln;
end.

```

Шаг 5. Полный код

```

Program DeleteElements;
var
  X: array [1..50] of integer;
  i, j, n, k: integer;
Begin
  write('Введите количество элементов (не более 50): ');
  readln(n);
  writeln('Введите элементы массива:');
  for i := 1 to n do

```

```

    readln(X[i]);
write('Введите число k для удаления: ');
readln(k);

i := 1;
while i <= n do
begin
    if X[i] = k then
    begin
        for j := i to n-1 do
            X[j] := X[j+1];
        n := n - 1;
    end
    else
        i := i + 1;
    end;

writeln('Массив после удаления элементов ', k, ':');
if n = 0 then
    writeln('Массив пуст')
else
    for i := 1 to n do
        write(X[i], ' ');
    writeln;
    readln;
end.

```

Шаг 6. Проверь работу

Введи $n = 6$, массив: [4, 2, 5, 2, 8, 2], $k = 2$. Программа должна вывести: 4

5 8

Задание 4. Разбор алгоритма вставки элемента в массив

Шаг 1. Понимание задачи

Дан массив X из n элементов. Нужно вставить новый элемент на позицию m (между $m-1$ и m). После вставки массив будет содержать $n+1$ элемент.

Шаг 2. Почему сдвиг начинают с конца

Если начать сдвиг с начала, то элементы будут перезаписываться. Например, вставка на позицию 3 в массив [10, 20, 30, 40, 50]:

Неправильный сдвиг (слева направо):

- $X[3] = X[4] \rightarrow [10, 20, 40, 40, 50]$ (потеряли 30)
- $X[4] = X[5] \rightarrow [10, 20, 40, 50, 50]$
- Потеряли элемент 30, который нужно было сохранить

Правильный сдвиг (справа налево):

- $X[5] = X[4] \rightarrow [10, 20, 30, 40, 40]$
- $X[4] = X[3] \rightarrow [10, 20, 30, 30, 40]$
- $X[3] \leftarrow \text{новое значение} \rightarrow [10, 20, 99, 30, 40]$

Шаг 3. Общий алгоритм вставки на позицию m

Для вставки элемента value на позицию m в массиве размером n:

1. Сдвинуть элементы с индекса n вниз до m вправо (цикл от $i = n$ вниз до m)
2. Выполнить $X[m] = \text{value}$
3. Увеличить n на 1

Код цикла сдвига:

```
for i := n downto m do
  X[i+1] := X[i];
X[m] := value;
n := n + 1;
```

Шаг 4. Важное замечание о размере массива

При описании массива нужно предусмотреть запас места для вставки. Если исходный массив описан как array [1..50] of integer и $n = 50$, то вставить новый элемент не получится (выйдем за границы). Нужно либо описывать массив с запасом, либо проверять, что n меньше максимального размера.

Задание 5. Создай программу вставки элемента

Условие: Написать программу, которая запрашивает у пользователя массив из n целых чисел (n не более 40), затем запрашивает число для вставки и позицию для вставки. Программа вставляет элемент и выводит новый массив. Если позиция некорректна (от 1 до n+1), вывести сообщение об ошибке.

Выполни задание по шагам.

Шаг 1. Заголовок и переменные

Напиши:

```
Program InsertElement;
var
  X: array [1..50] of integer;
  i, n, pos, value: integer;
```

Шаг 2. Начало и ввод данных

Напиши:

```
Begin
  write('Введите количество элементов (не более 40): ');
  readln(n);
  writeln('Введите элементы массива:');
  for i := 1 to n do
    readln(X[i]);
  write('Введите число для вставки: ');
  readln(value);
  write('Введите позицию для вставки (от 1 до ', n+1, '): ');
  readln(pos);
```

Шаг 3. Проверка корректности позиции

Напиши:

```
if (pos < 1) or (pos > n+1) then
  writeln('Ошибка: неверная позиция')
else if n = 50 then
  writeln('Ошибка: массив полон')
else
  begin
```

Шаг 4. Сдвиг и вставка

Напиши:

```
  for i := n downto pos do
    X[i+1] := X[i];
  X[pos] := value;
  n := n + 1;
```

Шаг 5. Вывод результата

Напиши:

```
  writeln('Массив после вставки:');
  for i := 1 to n do
    write(X[i], ' ');
  writeln;
end;
```

```
  readln;
end.
```

Шаг 6. Полный код

Program InsertElement;

var

X: array [1..50] of integer;

i, n, pos, value: integer;

Begin

write('Введите количество элементов (не более 40): ');

readln(n);

writeln('Введите элементы массива:');

for i := 1 to n do

readln(X[i]);

write('Введите число для вставки: ');

readln(value);

write('Введите позицию для вставки (от 1 до ', n+1, '): ');

readln(pos);

```
  if (pos < 1) or (pos > n+1) then
    writeln('Ошибка: неверная позиция')
```

```

else if n = 50 then
  writeln('Ошибка: массив полон')
else
begin
  for i := n downto pos do
    X[i+1] := X[i];
  X[pos] := value;
  n := n + 1;

  writeln('Массив после вставки:');
  for i := 1 to n do
    write(X[i], ' ');
  writeln;
end;

```

```

  readln;

```

```

end.

```

Шаг 7. Проверь работу

Введи $n = 5$, массив: [10, 20, 30, 40, 50], $value = 99$, $pos = 3$. Программа должна вывести: 10 20 99 30 40 50

Задание 6. Разбор пузырьковой сортировки из лекции

Возьмём программу сортировки из лекции.

Код программы:

```

program p8_2;
const
  n = 10;
var
  a: array [1..n] of Real;
  i, j : integer;
  temp : Real;
begin
  Writeln('введите ', n, ' элементов массива:');
  for i := 1 to n do Read(A[i]); Readln;

  Writeln('Массив до сортировки:');
  for i:= 1 to n do Write(A[i]:6:2, ' ');
  Writeln;

  For j := 1 to n-1 do
    For i := 1 to n - j do
      if A[i] > A[i+1] then
        begin
          temp := A[i+1];
          A[i+1] := A[i];

```



```
A[i] := temp;  
end;
```

```
Writeln('Массив после сортировки:');  
for i:= 1 to n do Write(A[i]:6:2, ' ');  
Writeln;  
end.
```

Шаг 1. Почему она называется «пузырьковой»

Название происходит от образной интерпретации: более «лёгкие» (меньшие) элементы «всплывают» в начало массива, как пузырьки в воде, а более «тяжёлые» (большие) опускаются в конец.

Шаг 2. Как работает алгоритм

Алгоритм сравнивает соседние элементы. Если левый элемент больше правого, они меняются местами.

Проход 1 ($j=1$): сравниваются пары от $i=1$ до $n-1$

- Сравниваем $A[1]$ и $A[2]$: если $A[1] > A[2]$, меняем
- Сравниваем $A[2]$ и $A[3]$: если $A[2] > A[3]$, меняем
- ...
- Сравниваем $A[n-1]$ и $A[n]$: если $A[n-1] > A[n]$, меняем

После первого прохода самый большой элемент оказывается в конце (на позиции n).

Проход 2 ($j=2$): сравниваются пары от $i=1$ до $n-2$ (последний элемент уже на месте)

Проход 3 ($j=3$): сравниваются пары от $i=1$ до $n-3$
И так далее.

Шаг 3. Пример работы

Массив: [5, 2, 8, 1, 3]

Проход 1 ($j=1$, i от 1 до 4):

- $i=1$: $5 > 2$? Да → меняем → [2, 5, 8, 1, 3]
- $i=2$: $5 > 8$? Нет → [2, 5, 8, 1, 3]
- $i=3$: $8 > 1$? Да → меняем → [2, 5, 1, 8, 3]
- $i=4$: $8 > 3$? Да → меняем → [2, 5, 1, 3, 8]

Результат: 8 всплыл в конец

Проход 2 ($j=2$, i от 1 до 3):

- $i=1$: $2 > 5$? Нет → [2, 5, 1, 3, 8]
- $i=2$: $5 > 1$? Да → меняем → [2, 1, 5, 3, 8]
- $i=3$: $5 > 3$? Да → меняем → [2, 1, 3, 5, 8]

Результат: 5 всплыл на своё место

Проход 3 ($j=3$, i от 1 до 2):

- $i=1$: $2 > 1$? Да → меняем → [1, 2, 3, 5, 8]
- $i=2$: $2 > 3$? Нет → [1, 2, 3, 5, 8]

Результат: массив отсортирован

Шаг 4. Почему внешний цикл от 1 до $n-1$

Внешний цикл for j := 1 to n-1 do означает, что нужно сделать n-1 проходов. Почему не n? Потому что если n-1 элемент стоит на своём месте, то последний (n-й) автоматически тоже на своём месте.

Шаг 5. Почему внутренний цикл от 1 до n-j

Внутренний цикл for i := 1 to n-j do с каждым проходом становится короче на один элемент. Это логично, потому что самые большие элементы уже заняли свои места в конце массива и их не нужно проверять снова.

Задание 7. Создай программу сортировки самостоятельно

Условие: Написать программу пузырьковой сортировки для массива целых чисел. Количество элементов n вводит пользователь (не более 50). Программа выводит массив до и после сортировки как по возрастанию, так и по убыванию.

Шаг 1. Заголовок и переменные

Напиши:

```
Program BubbleSort;
var
  A: array [1..50] of integer;
  i, j, n, temp: integer;
```

Шаг 2. Начало и ввод данных

Напиши:

```
Begin
  write('Введите количество элементов (не более 50): ');
  readln(n);
  writeln('Введите элементы массива:');
  for i := 1 to n do
    readln(A[i]);
```

Шаг 3. Вывод исходного массива

Напиши:

```
writeln('Исходный массив:');
for i := 1 to n do
  write(A[i], ' ');
writeln;
```

Шаг 4. Сортировка по возрастанию

Напиши:

```
for j := 1 to n-1 do
  for i := 1 to n-j do
    if A[i] > A[i+1] then
      begin
        temp := A[i];
        A[i] := A[i+1];
```

```
    A[i+1] := temp;  
end;
```

Шаг 5. Вывод отсортированного массива

Напиши:

```
writeln('Массив по возрастанию:');  
for i := 1 to n do  
    write(A[i], ' ');  
writeln;
```

Шаг 6. Завершение

Напиши:

```
    readln;  
end.
```

Шаг 7. Полный код

```
Program BubbleSort;  
var  
    A: array [1..50] of integer;  
    i, j, n, temp: integer;  
Begin  
    write('Введите количество элементов (не более 50): ');  
    readln(n);  
    writeln('Введите элементы массива:');  
    for i := 1 to n do  
        readln(A[i]);  
  
    writeln('Исходный массив:');  
    for i := 1 to n do  
        write(A[i], ' ');  
    writeln;  
  
    for j := 1 to n-1 do  
        for i := 1 to n-j do  
            if A[i] > A[i+1] then  
                begin  
                    temp := A[i];  
                    A[i] := A[i+1];  
                    A[i+1] := temp;  
                end;  
            end;  
        end;  
  
    writeln('Массив по возрастанию:');  
    for i := 1 to n do  
        write(A[i], ' ');  
    writeln;
```

```
readln;  
end.
```

Шаг 8. Как сделать сортировку по убыванию

Для сортировки по убыванию достаточно изменить знак в условии:

```
if A[i] < A[i+1] then
```

Это будет менять местами элементы, если левый меньше правого, то есть большие будут «всплывать» влево.

Самостоятельные задачи к Практике 7

Задачи на удаление элементов (3 задачи)

Задача У1. Удаление всех чётных элементов

Напиши программу, которая запрашивает у пользователя массив из n целых чисел и удаляет из него все чётные элементы. Выведи получившийся массив.

Ожидаемый вывод:

Введите количество элементов: 6

Введите элементы: 3 6 2 7 4 9

Массив после удаления чётных: 3 7 9

Задача У2. Удаление элемента по индексу

Напиши программу, которая запрашивает у пользователя массив и индекс элемента, который нужно удалить (пользователь вводит позицию от 1 до n). Программа удаляет элемент с указанным индексом и выводит новый массив. Если индекс некорректен, вывести сообщение об ошибке.

Ожидаемый вывод:

Введите элементы: 10 20 30 40 50

Введите индекс для удаления: 3

Массив после удаления: 10 20 40 50

Задача У3. Удаление повторяющихся элементов (оставить уникальные)

Напиши программу, которая оставляет в массиве только уникальные элементы (удаляет все повторяющиеся вхождения). Например, из массива [1, 2, 2, 3, 3, 3, 4] должно получиться [1, 2, 3, 4].

Подсказка: для каждого элемента проверь, встречается ли он раньше. Если да — удали.

Задачи на вставку элементов (3 задачи)

Задача В1. Вставка в начало и в конец

Напиши программу, которая запрашивает у пользователя массив, затем вставляет заданное число в начало массива и другое число в конец массива. Выведи получившийся массив.

Ожидаемый вывод:

Введите элементы: 5 10 15 20

Введите число для вставки в начало: 0

Введите число для вставки в конец: 25

Массив: 0 5 10 15 20 25

Задача В2. Вставка после первого вхождения

Напиши программу, которая вставляет заданное число после первого вхождения другого заданного числа. Например, в массиве [1, 2, 3, 2, 4] после числа 2 вставляем 99 → получается [1, 2, 99, 3, 2, 4]. Если число не найдено, вывести сообщение.

Задача В3. Вставка нескольких элементов

Напиши программу, которая запрашивает у пользователя массив, затем второе число — позицию, с которой нужно начать вставку, и третье число — количество вставляемых элементов. Пользователь вводит все вставляемые числа. Программа вставляет их в указанную позицию по очереди (каждая следующая вставка сдвигает остальные).

Ожидаемый вывод:

Исходный массив: 10 20 30 40

Позиция для вставки: 2

Сколько элементов вставить: 3

Введите элементы для вставки: 11 22 33

Результат: 10 11 22 33 20 30 40

Задачи на сортировку (3 задачи)

Задача С1. Сортировка по убыванию

Напиши программу, которая сортирует массив по убыванию (от большего к меньшему) с помощью пузырьковой сортировки.

Ожидаемый вывод:

Введите элементы: 3 7 1 9 4

Отсортировано по убыванию: 9 7 4 3 1

Задача С2. Сортировка с подсчётом перестановок

Напиши программу пузырьковой сортировки, которая дополнительно подсчитывает, сколько перестановок элементов было сделано за всю сортировку. Выведи отсортированный массив и количество перестановок.

Ожидаемый вывод:

Введите элементы: 5 2 8 1 3

Отсортировано: 1 2 3 5 8

Сделано перестановок: 6

Задача С3. Сортировка только чётных элементов

Напиши программу, которая сортирует по возрастанию только чётные элементы массива, оставляя нечётные на своих местах. Например, массив [5, 4, 3, 8, 2] → чётные элементы: 4, 8, 2. Сортируем их: 2, 4, 8. Вставляем обратно на позиции чётных → [5, 2, 3, 4, 8].

Комбинированные задачи (3 задачи)

Задача К1. Удаление и сортировка

Напиши программу, которая сначала удаляет из массива все элементы меньше заданного числа, а затем сортирует оставшиеся элементы по возрастанию.

Ожидаемый вывод:

Введите элементы: 10 3 5 8 2 7 9

Удаляем элементы меньше 5

После удаления: 10 5 8 7 9

Отсортировано: 5 7 8 9 10

Задача К2. Вставка с сохранением порядка

Напиши программу, которая вставляет заданное число в массив так, чтобы после вставки массив оставался отсортированным по возрастанию. Изначальный массив уже отсортирован.

Ожидаемый вывод:

Исходный отсортированный массив: 10 20 30 40 50

Введите число для вставки: 25

Результат: 10 20 25 30 40 50

Задача К3. Слияние двух массивов

Напиши программу, которая запрашивает два отсортированных по возрастанию массива и объединяет их в третий отсортированный массив (без сортировки результата — используй алгоритм слияния).

Ожидаемый вывод:

Массив А (отсортирован): 1 3 5 7

Массив В (отсортирован): 2 4 6 8

Результат слияния: 1 2 3 4 5 6 7 8

Повышенной сложности (2 задачи)

Задача П1. Быстрая сортировка (рекурсивная)

Напиши программу, реализующую алгоритм быстрой сортировки (QuickSort). При рекурсивном подходе выбирается опорный элемент, массив делится на две части (меньшие опорного и большие), затем каждая часть сортируется рекурсивно.

Задача П2. Удаление нулей со сдвигом

Напиши программу, которая удаляет все нули из массива, при этом оставшиеся элементы сдвигаются влево, а в конце массива оставшиеся позиции заполняются -1. Например, $[0, 5, 0, 3, 0, 8] \rightarrow [5, 3, 8, -1, -1, -1]$.